BaiZe: A 2.2B Hybrid Language Model as an On-Premise EDA Subagent

Abstract—Agentic LLMs for electronic design automation (EDA) are already deployed in-house, with a frontier large language model acting as the orchestrator—the agent that faces the user, understands intent, decomposes the work, and invokes subagents. Cost, not feasibility, is now the binding constraint: every EDA subtask routed to the orchestrator consumes tokens and serving budget that scale with design-flow volume. Privacy compounds the constraint: netlists, GDSII layouts, and design rules are trade secrets that must not leave the premises, so the on-site subtask is served by a small local model or not at all. We present BaiZe, a ≈2.2B hybrid state-space agentic model built from scratch to serve as a small, local subagent beneath the orchestrator: it translates the orchestrator's intent into native EDA scripts (PyAether, SKILL, Tcl) and operates each tool. Its 56 layers interleave Mamba-2 state-space and MLP layers with four attention layers, so only the latter carry a session-growing state; decoding is linear-time and per-subtask serving cost is governed by session memory rather than prompt throughput. A matched from-scratch comparison against a dense transformer selects this architecture (11.6% fewer parameters, lower loss at a matched 1k-step budget, and $+10.5\%$ prefill throughput), and a budget-constrained search fixes the pre-training recipe, now scaled to a 2.94T-token production run. The backbone now stands at training loss 2.151 after 30,660 iterations (≈598K tok/s). On a 449-task EDA-scripting benchmark spanning three languages and four commercial runtimes [1], we test whether BaiZe can offload subtasks from the deployed orchestrators at comparable task accuracy and materially lower serving cost. Later stages are specified as protocols with pre-specified criteria; their measurements are in progress, and we release the full recipe and evaluation protocol.

Index Terms—small language model, agentic LLM, hybrid state-space architecture, EDA subagent, cost-efficient inference

I. Introduction

Large language models have recently demonstrated agentic capabilities—planning, tool invocation, and long-horizon reasoning—that make them attractive cores for intelligent assistants in electronic design automation (EDA). Such assistants are no longer hypothetical: agentic EDA flows are already deployed in-house, with a frontier large language model serving as the orchestrator that coordinates the flow. The binding constraint is therefore cost, dominated by two terms: every subtask routed to the orchestrator consumes tokens and serving budget that scale with flow volume, and because an agentic session accumulates context, serving cost is driven by the memory that grows with the session rather than by prompt throughput. A small, locally served subagent caps the cost of every subtask it takes over by pricing it at a local 2.2B model rather than at an orchestrator. This is the metaphor of the Artifact Spirit (器灵): the orchestrator stays in the cloud to hold the user's intent and long-horizon plan, while every EDA tool is paired with its own small, local spirit that translates intent into tool-native scripts and operates the tool. Privacy compounds the cost argument: netlists, GDSII layouts, and design rules are trade secrets that may not leave the premises, so for on-site subtasks the choice is between a small local model and no model at all. Building such a model from scratch as a production subagent, however, remains an under-documented engineering problem.

a) Contributions.: We present BaiZe, a ≈2.2B-parameter hybrid (state-space + attention) agentic model built from scratch to serve as a cost-efficient EDA subagent beneath a deployed orchestrator.

- A cost-efficient EDA subagent. BaiZe receives EDA subtasks in natural language from the orchestrator and drives EDA tools in their native scripting languages (PyAether, SKILL, Tcl). Its 56 layers interleave Mamba-2 state-space and MLP layers with four grouped-query-attention layers, so only the latter carry a session-growing state—which is what makes per-subtask cost reduction structural rather than a matter of quantization alone. A staged budget-constrained search (S1–S5) with a reduced-horizon proxy protocol fixes the pre-training recipe. On a 449-task EDA-scripting benchmark spanning three languages and four commercial runtimes, we ask whether it matches deployed orchestrators at comparable accuracy while materially reducing per-subtask cost, and what an on-premise subagent must co-locate with the model.

II. Background and Related Work

A. Small Language Models and Agentic Reasoning

Parameter-efficient SLMs ($< 2\,\mathrm{B}$) have recently narrowed the gap with larger models on mathematical and commonsense reasoning. MiniCPM [2] introduced scalable training strategies and domain-weighted data mixing for small models; Qwen [3] and Gemma [4] demonstrate that strong code- and math-enriched corpora can lift small models substantially. Most of these efforts, however, treat the final trained model as the deliverable and report comparatively little about the search that produced it. Complementarily, agentic instruction-tuning datasets (e.g., tool-use and multi-step reasoning corpora) [5] have been shown to elicit planning and tool-invocation behaviour, yet are rarely paired with a from-scratch pre-training recipe for the 1B regime.

B. Hybrid and State-Space Architectures

State-space models (SSMs) such as Mamba [6] and Mamba-2 [7] replace full attention with a recurrent scan update, offering linear-time decoding that suits inference-bound deployments. Hybrid designs interleave a small number of

attention layers among SSM/MLP layers—notably Nemotron-H [8] and Hymba [9]—to recover long-range context while retaining fast decode. Released Megatron-Core variants of these families are scarce below 2B, leaving their small-scale trade-offs under-documented; our controlled 2B comparison (Section IV-A) addresses precisely this gap.

C. LLMs for Electronic Design Automation

A growing line of work applies LLM agents to chip design tasks such as register-transfer-level generation, verification, and bug triage [10], [11]. These applications motivate a cheap, always-on, agentic core that can run per-team or per-engineer; our work supplies the efficient backbone such systems require.

III. Architecture and Pipeline Overview

BaiZe is an agentic system assembled in two stages: (i) from-scratch pre-training of a hybrid language backbone, and (ii) post-training of that backbone (instruction tuning and reinforcement learning to strengthen agentic capability). At deployment, BaiZe is a subagent in a star topology: a central orchestrator holds the user's intent and dispatches each subtask to the subagent of the relevant EDA tool, which translates the subtask into that tool's native script (PyAether, SKILL, Tcl) and operates the tool locally. This section fixes the shared scaffolding—the backbone, the overall pipeline, and the global training environment—before each stage is detailed in its own section.

A. Language Backbone

The language backbone is a sequential hybrid model [7], [8]: 56 transformer layers interleave Mamba-2 state-space layers, four grouped-query-attention (GQA) layers, and MLP layers. The layer allocation follows the Nemotron-H sequential hybrid convention—four attention layers among the 56, the remainder Mamba-2 and MLP—as provided by Megatron-Core's Mamba-hybrid recipe, whose 56-layer pattern we reuse verbatim while halving only the hidden and FFN widths to reach the ≈2B scale. BaiZe is not a Nemotron-H model: it is an order of magnitude smaller, contains no mixture-of-experts layers, and is pre-trained from scratch on an independent corpus under a fixed GPU-hour budget. Each Mamba-2 layer uses 8 state-space groups; the four attention layers use 16 query heads and 4 query groups (GQA) with untied word embeddings. Crucially, only the four attention layers carry rotary position embeddings (RoPE); the 52 non-attention layers (24 Mamba-2 SSM and 28 MLP) are position-agnostic, so the backbone's long-context extrapolation quality depends on just $4/56{=}7.1\%$ of layers, an asymmetry exploited in Section IV-H. The full configuration is listed in Table I; the model totals ≈2.22B parameters.

B. Pipeline and Global Environment

Table II summarises the two-stage pipeline. Stage (i) production training runs on two $8\times$NVIDIA H100 nodes (16 GPUs; CUDA 12.8, PyTorch 2.8.0, Python 3.10); other stages use a single $8\times$H100 node. Fused kernels, selective activation recomputation, and cross-entropy loss fusion are enabled where applicable, with `tensor_model_parallel_size` $= 1$; Stage (i) trains in FP8 tensorwise at the main sequence length, reverting to BF16 at shorter lengths.

TABLE I
BaiZe language-backbone configuration (Mamba2-hybrid).

Hyper-parameter	Value
Transformer layers	56
Architecture	hybrid (Mamba-2 + GQA + MLP)
Mamba-2 / MLP layers	24 / 28
Attention layers	4 (interleaved)
Hidden size	2048
FFN hidden size	8192
Attention heads (Q)	16
Query groups (GQA)	4
Mamba-2 state-space groups	8
Vocabulary size	129,281
Sequence length	16,384
Activation	SiLU / SwiGLU
Normalisation	RMSNorm
Precision	FP8 tensorwise
Parameters	≈2.22B

TABLE II
BaiZe two-stage pipeline overview.

Stage	Task	Status	Section
(i)	LM pre-training	in progress	IV
(ii)	Post-training (SFT, RLVR)	design	V

IV. LLM Pretraining

Stage (i) pre-trains the hybrid backbone (Section III-A) from scratch under a fixed budget.

A. Architecture Selection

We adopt the hybrid backbone because a matched from-scratch comparison at ≈2B scale favours it over a dense transformer (MiniCPM5-2B [12]): lower loss with 11.6% fewer parameters (≈2.22B vs. 2.51B) at ∼8% throughput cost (Table V); the Megatron-Core provider (Section III-A) makes it executable without custom kernels.

B. Training Framework and Learning-Rate Schedule

We use NVIDIA NeMo [13] with Megatron-Core's [14] Mamba-hybrid provider (Section III-A); hyper-parameter searches run at a reduced iteration count with short-horizon ranking transferred to full scale.

BaiZe trains under a WSD schedule (LR 1×10^{-3}, 2,000 warmup $= 0.29\%$ of 700k; search found 250 optimal at 5k but the proportion narrows at full scale) at sequence 16,384 (613,992 tok/s; $64\times$ extrapolation to 1M). Decay has two segments: A anneals at 16K on instruction data (60–70%), B extends on a 32K→64K→128K ladder (30–40%), with $lr_{\min}{=}1 \times 10^{-4}$.

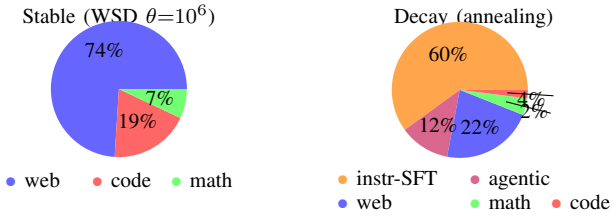


Fig. 1. Data mixture of the stable (WSD θ=10^6) and decay (annealing) phases. The decay segment replaces the bulk of general web/code/math with instruction-SFT (60%) and agentic-SFT (12%) data.

TABLE III
Long-context data gate (left) and training ladder (right). Per-GPU tokens held at 16,384; 32K throughput measured, 64K/128K extrapolated.

Thresh.	Docs	Share	Status	Rung	Seq.	CP/DP	Tput
≥32K	4,257	2.52%	not met	1	32K	2/8	351,526
≥128K	—	0.03%	not met	2	64K	4/4	≈180K
≥1M	—	n/a	n/a	3	128K	8/2	≈90K

C. Search Protocol

The recipe was selected via a staged single-variable protocol at 5000 steps: S1 baseline; S2 LR sweep (2×10^{-4}–1×10^{-3}, seven points); S3 schedule axes (WSD vs. cosine, decay ratio, warmup, min_lr); S4 annealing mix; S5 long run + three seeds. Total budget ≈82 GPU-hours on eight H100s; losses are proxy measurements for ranking.

D. Data

The stable phase uses a seven-source mixture yielding a 74:19:7 web/code/math split; a 400-trial Bayesian search confirmed this region optimal within 0.6 pp. Figure 1 shows the stable and decay (annealing) mixtures side by side; the decay segment drops most general web/code/math in favor of instruction-SFT and agentic-SFT data. Sources are pre-blended into large shards ($\mathcal{O}(10)$ vs. $\mathcal{O}(600)$ index files per rank), bounding the data-loader footprint. Contamination is controlled by a 193,295-entry 13-gram blacklist from 536 benchmarks; shards are scanned for ≥0.80 n-gram overlap.

E. Long-Context Data Gate

Only **2.52**% of tokens sit in documents ≥32K and 0.03% at ≥128K (mean 889 tokens; Table III), so we measured the distribution before budgeting segment B. We gate the ladder: ≥32K share must reach 20% and ≥128K 5% (else it stops at 64K), with EOD resets and de-duplication.

F. Results and Production Run

The search selects WSD at LR 1×10^{-3} with 86:10:4 annealing, reaching validation loss **2.2054** at 20000 steps (2.6739 ± 0.0469 across three seeds at 5000; Table V).

The recipe is scaled to 2.94T tokens at `DP`=16 over two 8×H100 nodes, `GBS`=256 at sequence 16,384 (4,194,304 tokens/step), FP8 tensorwise, AdamW. At the time of writing, the run has passed iteration 30,660 of 700,000 at ≈598K tok/s, training loss 2.151, no NaN (Table IV). The 613,992 tok/s peak is FP8 at 16,384; shorter sequences use BF16.

TABLE IV
Production budget. Wall-clock is estimated at measured throughput.

Phase	Tokens	Steps	Wall-clock	Status
Warmup + stable	2.94T	700k	≈56 d	30.7k done
Decay A (16K, 60–70%)	58.7B	14k	≈1.2 d	queued
Decay B (LC, 30–40%)	25.2B	6k	≈2.3 d	gated
Total	3.02T	720k	≈59.5 d	—

G. Long-Context Training in the Decay Tail

Long-context training is segment B of the decay: the LR is already low, and neither SFT nor RL builds retrieval ability at this scale. The decay budget is ∼2.9% of main volume (83.9B / 20,000 steps); long-context data is 30–40% (25–34B). The ladder runs 32K→64K→128K at `CP` ∈ {2, 4, 8}, 16,384 tokens/GPU, continuing decay without re-warming; θ=10^6 fixed, so 128K is trained and 1M is 8× extrapolation (Table III). Reference cost ∼880 GPU-hours (Table IV). At WSD end: one snapshot—needle retrieval (passkey + RULER, target ≥95% at 1M) and perplexity extrapolation (≤ 1.10×).

H. Long-Context Capability and Inference Cost

With θ=10^6 on the four attention layers, perplexity evolves 83.2 → 51.5 → 53.3 → 53.0 → 55.8 → 55.4 from 4K to 1M, flat beyond 64K; only 4/56=7.1% of layers carry RoPE. Under SGLang, the hybrid achieves 39,377 tok/s prefill at 128K and serves 1M in 172s at 34.7 GB; the dense transformer cannot serve beyond 512K on a single 80 GB H100 (KV-pool cap ≈455K). The advantage is architectural: $O(1)$ per-layer state for 52 of 56 layers. While profiling the hybrid under a CPU-only `llama.cpp` backend, we isolated and fixed an upstream inference-correctness bug for `Nemotron-H` (missing RoPE on attention layers and a wrong FFN activation), contributed as a fix to llama.cpp[1]; the fix restores 1-token logits to $\Delta_{\max}$=0.006 against the HF reference.

TABLE V
Architecture comparison (top; from scratch, matched data/seed/GPU) and search outcomes (bottom; 5000-step horizon, S5: 20000).

Metric	Dense	Hybrid	Edge
Parameters	2.51B	2.22B	hybrid (−11.6%)
Final loss (1k)	4.81	4.68	hybrid (−0.12)
Train loss (5k)	3.36	3.06	hybrid (−0.30)
Training tok/s	90K	83K	dense (+8%)
Prefill tok/s	21.4K	23.7K	hybrid (+10.5%)
Decode tok/s	2.2	22.8	hybrid (+10.3×)

ID	Change vs. best	Val. loss	Outcome
S1-01	baseline (lr=3×10^{-4}, WSD)	2.849	baseline
S2-07	stable LR = 1×10^{-3}	2.7627	best LR
S3-02	cosine	2.8155	WSD retains
S3-03	decay ratio 5%	2.7900	10% retains
S3-04	warmup 2000	2.7957	250 retains
S4-03	anneal 86:10:4	2.6298	best mix
S5-01	winner, 20000 steps	2.2054	winner

[1]https://github.com/ggml-org/llama.cpp/pull/31703

V. LLM Post-Training

Stage (ii) adapts the pre-trained backbone to EDA scripting, and it is the stage that carries BaiZe's domain claim: the pre-training corpus of Section IV-D is general web, code, and mathematics text and contains no EDA artifacts, so the domain prior must be introduced here. We organise the stage as three steps—domain annealing, supervised fine-tuning, and reinforcement learning—all evaluated under a single frozen protocol (Section V-E).

A. Domain Annealing

The cheapest lever on the domain prior is to continue pre-training on EDA text before any behavioural tuning. We reuse the warmup–stable–decay machinery of Stage (i) and anneal on a corpus assembled from PyAether and SKILL API references, EDA tool documentation, and open HDL sources, blended with the general mixture at a controlled ratio. Because the step changes only the data distribution, we contrast it against an SFT-only control to isolate its contribution from the behavioural tuning that follows.

B. Supervised Fine-Tuning

SFT teaches the generate–execute–refine loop that the deployment target requires. The corpus combines the agentic instruction set (`UltraData-SFT-Agent-2609`, 51 G across 50 shards) and a general instruction set (`UltraData-SFT-2605`, 319 G across 1,504 JSONL files, verified byte-identical to the HuggingFace manifest) with PyAether-specific material synthesised from API documentation. We ablate the data mixture (agentic / code / mathematics / PyAether), the presence of interleaved tool-call traces, full fine-tuning versus LoRA, and the sequence length: agentic traces—code, tool returns, and successive refinements—routinely exceed the 16,384-token window of Table I, so window extension is treated as a first-class variable rather than a fixed setting.

C. Reinforcement Learning with Verifiable Rewards

An execution sandbox turns correctness into a verifiable signal, so we optimise with group-relative policy optimisation and no value network. Table VI separates the reward variants we study. A binary signal is the simplest baseline; partial credit over executable attempts addresses the sparsity of pass/fail; a tool-call penalty aligns the policy with the harness cost model; and an API-legality term penalises hallucinated symbols. Because the reward is computed from assertions a policy could in principle target directly, we keep the harness anti-cheat hook enabled in every run and report a cheat rate beside every accuracy figure.

TABLE VI
Reward variants studied in stage (ii).

ID	Reward
R1	binary pass/fail
R2	executable + assertion pass ratio
R3	R2 + tool-call penalty
R4	R3 + API-legality term

D. Deployment: The Anima Protocol and Local Self-Correction

Deployed as a subagent, BaiZe is driven not by free-form prose but by a semi-structured anima directive that carries an action, a target object, and a constraint—e.g. {`action: route, target: net_clk, constraint: minimize_crosstalk`}. A deterministic directive removes the ambiguity of free-form commands, so script generation is conditioned on a narrow, tool-specific intent rather than on the whole session—this is what keeps the subagent small and its per-subtask cost low.

The subagent then runs a closed self-correction loop entirely on the client: it emits a script, executes it silently in the local tool sandbox, and, on failure, feeds the traceback back to itself to re-write until the script runs—without consulting the orchestrator.

E. Evaluation Protocol

We track four capability milestones through Stage (ii): M1 tool-call legality (syntactically valid PyAether calls, no hallucinated symbols); M2 single-task pass (Pass@1 on EDA-Eval-PyAether); M3 multi-turn refinement (recovery from execution failures with up to 3 rounds); M4 full-benchmark accuracy. All variants are scored on EDA-Eval-PyAether [1] (158 real-world EDA scripting tasks), each pairing a natural-language prompt and function signature with assertion-based test code, scored by execution in the vendor sandbox. We follow the benchmark's own protocol—five independent runs, reshuffled order, fresh context, mean Pass@1 with run-to-run standard deviation, 2500-s per-task timeout—so numbers are comparable to published ones. Two controls make the comparison interpretable: contamination is excluded by the n-gram blacklist of Section IV-D, and only the backbone is varied while the retrieval stack, sandbox, anti-cheat hook, and agent scaffold are held byte-identical, since the harness supplies most of the observed accuracy.

Results are reported as [TBD].

VI. Conclusion

We presented BaiZe, a ≈2.2B-parameter hybrid (state-space + attention) language model built from scratch and post-trained for agentic capability. Through a controlled from-scratch comparison, we selected a sequential Mamba2-hybrid backbone over a dense transformer for its lower loss-per-parameter and substantially faster inference, then applied a staged, budget-constrained hyper-parameter search (S1–S5) on the final backbone. Stage (ii) is specified here as a protocol and evaluated on a common execution-based benchmark [1]; its measurements are reported as [TBD]. The complete recipe and a single reproducible command encapsulate the entire construction under the NVIDIA NeMo recipe.

The immediate next step is agentic fine-tuning with the `UltraData-SFT-Agent` corpus toward a deployable agent

core for EDA and embedded applications. We hope this blueprint lowers the barrier for cost-bounded teams building their own efficient agentic SLMs from scratch.

References

[1] Y. Liu, S. Hou, X. Chen, Y. Wang, S. Yuan, Q. Gan, S. You, F. Chen, W. Li, S. Hu, Y. Liu, E. Xia, X. Lu, H. Wang, F. Xu, and Y. Li, "Zhulong: Execution-grounded llm agent for eda scripting with offline api self-exploration," 2026.

[2] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao et al., "Minicpm: Unveiling the potential of small language models with scalable training strategies," arXiv preprint arXiv:2404.06395, 2024.

[3] A. Yang, B. Li, B. Zhang et al., "Qwen3 technical report," 2025.

[4] G. Team, "Gemma 3 technical report," 2025.

[5] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang et al., "Agentbench: Evaluating llms as agents," in International Conference on Learning Representations (ICLR), 2024.

[6] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," arXiv preprint arXiv:2312.00752, 2023.

[7] T. Dao and A. Gu, "Transformers are ssms: Generalized models and efficient algorithms through structured state space duality," arXiv preprint arXiv:2405.21060, 2024.

[8] NVIDIA, "Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models," 2025.

[9] X. Dong, Y. Fu, S. Diao et al., "Hymba: A hybrid-head architecture for small language models," 2024.

[10] M. Liu, T. E. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu et al., "Chipnemo: Domain-adapted llms for chip design," in Advances in Neural Information Processing Systems (NeurIPS), 2024.

[11] J. Blocklove, R. Chapman, B. Tan, D. Chen et al., "Chipbench: The llm chip design benchmark," 2024.

[12] OpenBMB, "Minicpm5: Openbmb small language model series," https://github.com/OpenBMB/MiniCPM, 2026, model release (1B: 2026-05, 2B: 2026-09); no independent training paper available at time of writing.

[13] NVIDIA, "Nemo: A toolkit for building ai applications using neural modules," https://github.com/NVIDIA/NeMo, 2024, version 2.x; used for pre-training via the NeMo run entry point.

[14] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," 2020.